Machine Learning Course - MiniProject I

Matin Moradi

Åbo Akademi University
Matin.Moradi@abo.fi

Introduction

The Smyril is a passenger and car ferry owned and operated by the Faroese transport company Strandfaraskip Landsins. She is the largest ferry in the fleet and the fifth vessel to carry the name Smyril, which is the Faroese word for merlin. The ferry takes 200 cars and 975 passengers.[1]

Energy consumption of ships like MS Smyril is an important topic because by predicting it, we can manage the expenses, reduce emissions, and improve energy efficiency. The ship has many sensors that capture different data from the speed to the location and the direction of the wind, from February to April 2010. Our goal is to use these data to predict the fuel consumption of the ship.

Figure 1: Smyril Line's primary workarea is travel and freight transport in the North Atlantic.

Loading the Data

First, we download the csv files which are all in a zip format file. We go through all the csv files and read them. Note that we have 20 different csv files, each of them belongs to one of the sensors, but all of them have two columns, timestep and the value. In this stage we did four main things:

1. Parse Latitude and Longitude: The latitude and longitude in our original CSVs were in "degrees and minutes" format with hemisphere letter, like this: 6132.9236N. We coverted them into decimal degrees, which is the standard numeric format for latitude and longitude. So the converted format of our example will be 61.548727.
2. Parse the Datetime: We converted the.NET ticks timestamp in the original CSVs to the standard pandas datetime format.
3. Resample and Combine Data Into One Dataframe: The challenge is that we have a different CSV file for each feature, and each sensor has recorded its value in a different rate. To merge all of these data and have a single dataframe to work with, we defined a fixed interval (1 minute) and took the median of all the values in that period and set that as the value for that interval. So finally we will have around 60 * 24 * 60 rows (two months in minutes) for nearly 2 months of data.

   (Maybe it is worth to mention that first, I tried to perform an outer join between all the files and then drop the NaN rows but it was computationally expensive and faced problem. Then, I changed my approach to the resampling that I explained earlier and was successful)
4. Gaps and Missing Values: There are periods that the sensor data is missing and will be present as NaN in the dataframe we created. To deal with this issue, we interpolate the values if the gap is small (less than 3 minutes), otherwise we need to drop the missing values.

Data Analysis and Processing

Target Variable

First, we defined the target variable as:

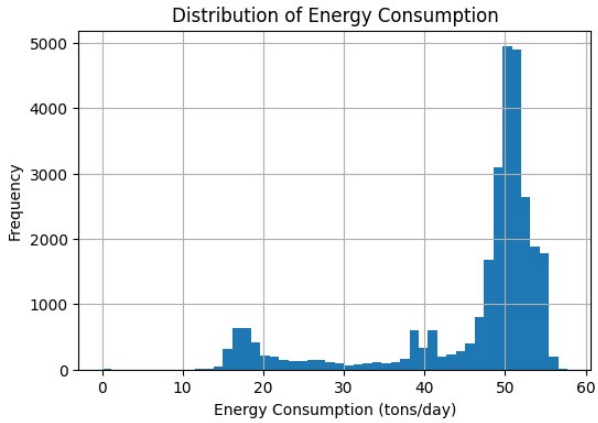
$$EC_{\text{tons_per_day}} = \frac{\text{fuelDensity} \cdot \text{fuelVolumeFlowRate} \cdot 3600 \cdot 24}{1000}$$

After defining the target variable, the columns `fuelDensity` and `fuelVolumeFlowRate` were removed from the feature set to prevent data leakage, since they are directly used to calculate the target.

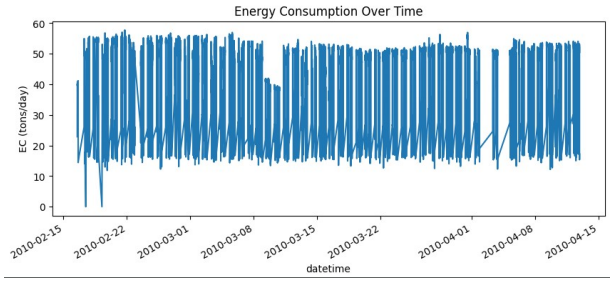
Figure 2 demonstrates the distribution of the target variable in our dataset and Figure 3 shows how the most correlated features relate to the target variable. We can see

[1] Wikipedia: MS *Smyril*

the nonlinear relationship between target and some features which gives some clues about the appropriate model to use.



(a) Histogram of EC_tons_per_day which shows the frequency distribution of the target variable



(b) Changes of EC_tons_per_day over time

Figure 2: EC_tons_per_day overview

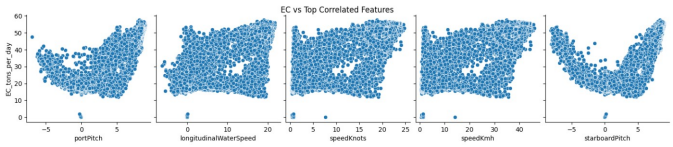


Figure 3: EC vs Top Correlated Features

Feature Engineering

In the next step, we look at the correlation. Figure 4 shows the pairwise correlation between some of the columns including the target label and Figure 5 shows a bar chart of feature pairs with absolute correlation greater than 0.8. The height of each bar shows the strength of the linear relationship, highlighting redundant features that may be candidates for removal. We can see that `speedKmh`, `speedKnots`, and `longitudinalWaterSpeed` are highly correlated because all three measure the same physical phenomenon (ship speed). So, we keep one of them (`speedKmh`) and drop others. Then we use speedKmh to define `speed_accel` feature by calculating the change of speed.

`portPitch` and `starboardPitch` are also two correlated features. we create two meaningful features from them:

$$\text{pitch}_{\text{mean}} = \frac{\text{portPitch} + \text{starboardPitch}}{2}$$

$$\text{pitch}_{\text{diff}} = |\text{portPitch} - \text{starboardPitch}|$$

Mean pitch represents propulsion load, while pitch difference captures asymmetry and inefficiency.

Also from `portRudder` and `starboardRudder` we can extract one feature that reflects both and then drop them:

$$\text{rudder_activity} = |\text{portRudder} - \text{starboardRudder}|$$

Rudder activity better reflects maneuvering intensity than absolute rudder angles.

Moreover, `trackDegreeTrue`, `trackDegreeMagnetic`, and `trueHeading` describe orientation and are highly correlated. So, we keep just one of them (trackDegreeTrue) and since the change in heading is important, we create the new feature `heading_change` out of it which is the difference between trackDegreeTrue values.

The wind angle is also converted into sine and cosine components to preserve its circular nature, so that angles near 0° and 360° are treated as close values by the model. This helps the model correctly use wind direction as a feature.

Finally, we selected and engineered 11 meaningful features + 1 target variable out of 20 raw columns.

Data Split and Scaling

In this last step before training, we split the data to use 80% of it for training and the remaining 20% for test and then scale it. We end up with 22936 rows for training and 5735 for test.

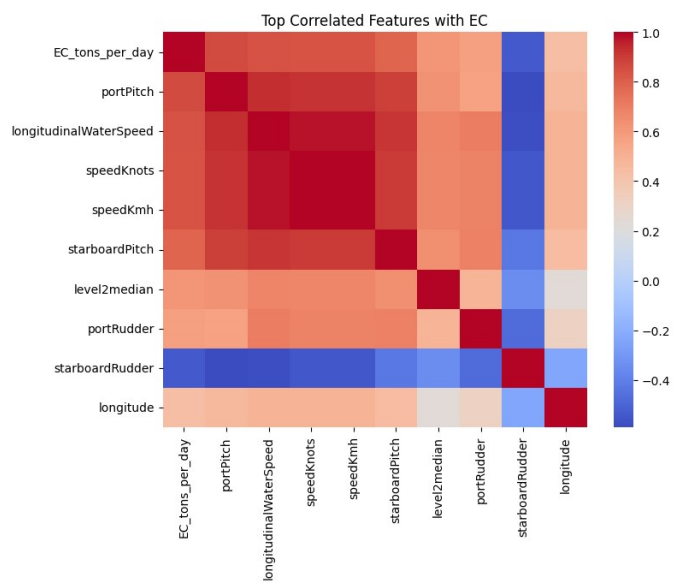


Figure 4: Top Correlated Features with EC

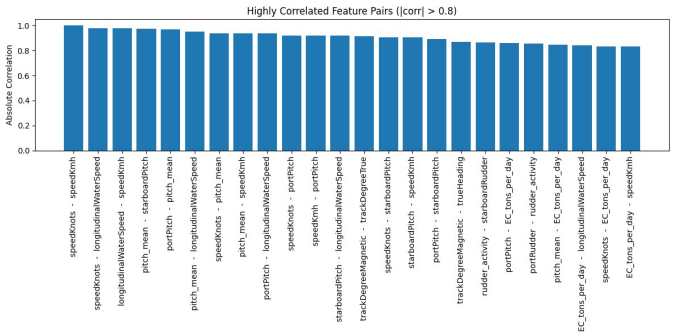


Figure 5: feature pairs with strong linear correlations

Model Training and Performance Evaluation

Three ML models are used. Linear Regression as a simple baseline and two other models which are supposed to perform better.

Linear Regression

A simple model that predicts the target as a weighted sum of input features, assuming a linear relationship between features and the target. The model achieved 80.8% accuracy on both the training and test sets, with `speedKmh` and `pitch_mean` having the largest coefficients. The details are in Table 1 and Table 2. Figure6 visually demonstrates the difference between the predicted and the actual labels.

Dataset	RMSE	R^2
Train	4.613	0.808
Test	4.580	0.808

Table 1: Performance metrics of the linear regression model.

Feature	Coefficient
speedKmh	5.049630
pitch_mean	4.557282
pitch_diff	2.700631

Table 2: Top 3 Feature coefficients of the linear regression model

Polynomial Linear Regression

The simple linear regression model didn't reach the required accuracy because as it was mentioned about Figure 3, some of the features have nonlinear relationship with the target variable and linear regression can not capture that. So, we change it a bit to make it polynomial of second degree. Polynomial linear regression extends linear regression by including polynomial combinations of features, allowing the model to capture nonlinear relationships while still remaining linear in the coefficients. Results are improved a lot and reached 97.6% for both the train and test sets. Table 3 and Figure 7 show the performance.

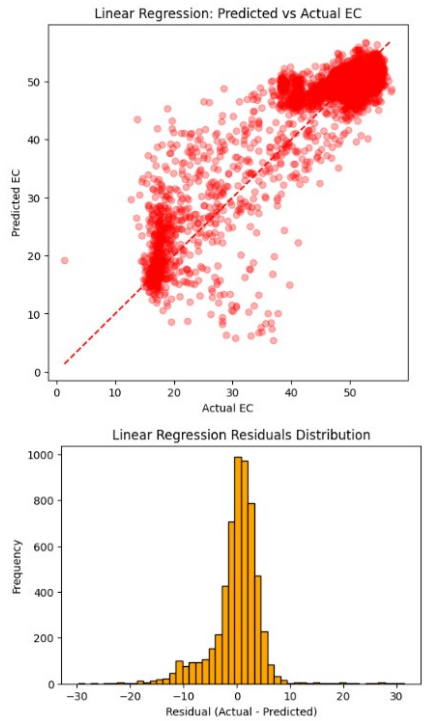


Figure 6: Visualization of Linear Regression Performance

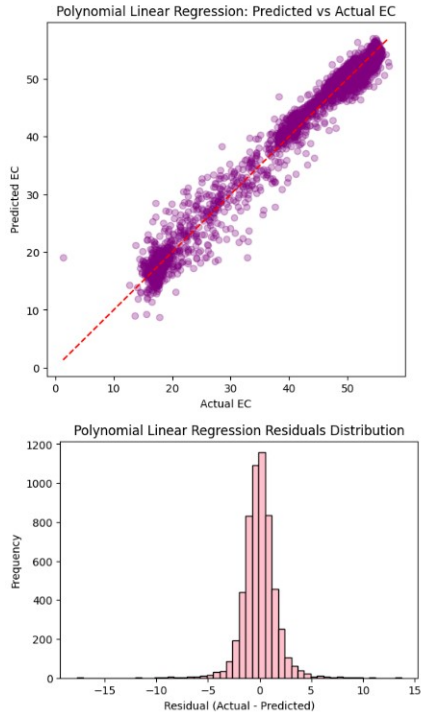


Figure 7: Visualization of Polynomial Linear Regression Performance

Dataset	RMSE	R^2
Train	1.617	0.976
Test	1.629	0.976

Table 3: Performance metrics of the polynomial linear regression model.

Random Forest

Random forest is the last model we used which is an ensemble of decision trees that predicts by averaging outputs from many trees, in this case 100 decision trees. To improve the performance and prevent overfitting we limit the growth of the trees. If a node has fewer than 5 samples, it becomes a leaf node, and no further splits occur there. The results were the highest using random forest and it reached 99.8% for training and 98.8% for test as described in Table4. The random forest model performs very well, with high accuracy, low error, and no overfitting.

Dataset	RMSE	R^2
Train	0.494	0.998
Test	1.164	0.988

Table 4: Performance metrics of the Random Forest regression model.

Figure8 demonstrates the difference between the predicted and true labels, which is better than the previous models. In Figure9 we did an experiment to find out how the model's performance changes as we use more training data. The blue line shows the R^2 score on the training set, and the orange line shows it on the test set. The gap between two curves is low and also both of the curves are increasing which means that there's no overfitting. The amount of current data is enough to get good results; however, the plot indicates that increasing the size of dataset can potentially improve the model's accuracy.

Conclusion

In this work, we predicted the energy consumption of the ship using sensor data. We cleaned and prepared the data, created new features, and selected the most important ones. The most challenging part was preparing the data for training. It was educational to work on a problem where the data was not in a single dataframe, but in different CSV files, each recording one feature at a different rate. As a first simple attempt, I tried to join all the tables, which failed and made me consider resampling the data at fixed time intervals.

We trained three models: linear regression as the baseline, polynomial linear regression and random forest as the improved solutions.

Polynomial linear regression was better than simple linear regression because it can capture nonlinear relationships between the features and the target. By adding polynomial terms, it models more complex patterns that a straight line cannot fit. Random Forest works better than polynomial linear regression because it can find nonlinear patterns and feature interactions automatically. It is also more robust to noise

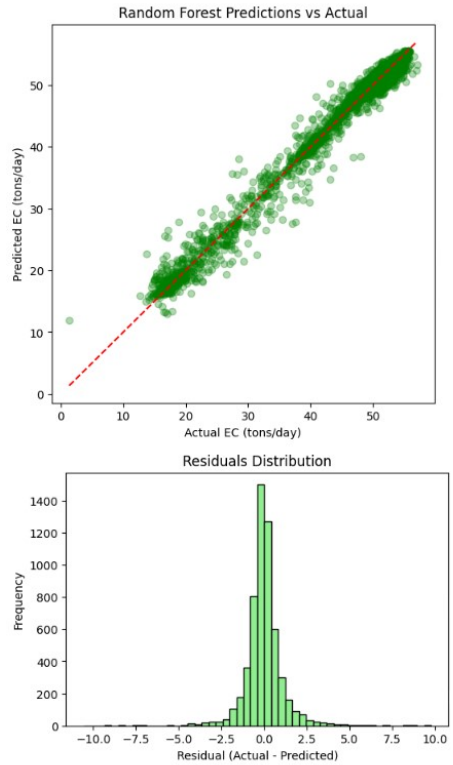


Figure 8: Visualization of Random Forest Performance

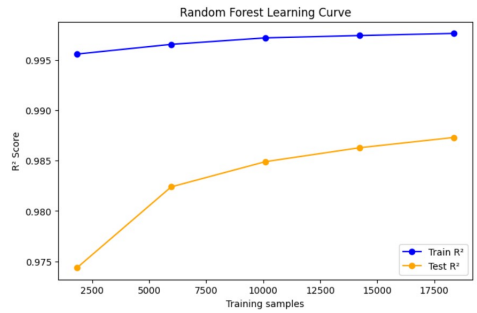


Figure 9: Random Forest - Learning Curve Plot

and outliers, while polynomial regression can overfit when using high-degree terms or when features are correlated. The ensemble of trees in Random Forest reduces overfitting and handles redundant or interacting features well, making it better for predicting the ship's energy consumption in our setup.

For the future, one can try shorter intervals than 1 minute for resampling in the preprocessing phase, which could potentially improve the results, as the learning curve suggested that adding more data can still be useful.